

DengAI — Predicting Disease Spread

Metrics, Models, and Notebook Preparation

October 2026

Contents

Part 1 — Metrics	1
Part 2 — Models	3
Part 3 — Notebook Preparation	5

Part 1 — Metrics

The competition metric is MAE

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

The average number of cases we are off by, per week.

- Reported over all **416 test weeks** (260 San Juan + 156 Iquitos)
- Lower is better; a perfect model scores 0
- Both cities are pooled into one number, so **San Juan dominates** — it has 63% of the test weeks *and* roughly four times the case counts

Why MAE and not RMSE

The target is extremely skewed:

	San Juan	Iquitos
mean cases/week	34.2	7.6
median	19	5
max	461	116
skewness	4.48	4.00

In both cities the **worst 10% of weeks hold 43% of all cases**.

Why MAE and not RMSE (cont.)

RMSE squares the errors, so a single epidemic week with 461 cases contributes as much as **~500 ordinary weeks** being off by one.

- Under RMSE the model is dragged into fitting a handful of outbreaks
- MAE weights every week equally
- Statistically: **MAE is minimised by the conditional median, RMSE by the mean** — and for a skewed count distribution the median is the more robust target

This also explains a behaviour we see later: good models here are *conservative*.

Two different MAEs — do not confuse them

	Cross-validation MAE	Holdout MAE
Computed on	5 expanding-window folds	one untouched 52-week block
Used for	choosing hyperparameters	comparing final models
Seen during tuning?	yes — the grid optimises it	no

CV MAE is the *tuning signal*. Holdout MAE is the *honest estimate*. They are never equal, and a large gap between them is itself information: it means the model is overfitting the folds.

Why validation must be chronological

Weekly case counts are **autocorrelated at 0.97** from one week to the next.

A random train/test split would therefore put week t in training and week $t + 1$ in test — the model effectively memorises the answer and the score becomes meaningless.

We use `TimeSeriesSplit(n_splits=5, test_size=52)`: always **train on the past, validate on the future**.

The fold structure

City	Fold	Train ends	Validation
San Juan	1	2002-04-23	2002-04-30 → 2003-04-23
San Juan	3	2004-04-22	2004-04-29 → 2005-04-23
San Juan	5	2006-04-23	2006-04-30 → 2007-04-23
Iquitos	1	2004-06-24	2004-07-01 → 2005-06-25
Iquitos	5	2008-06-24	2008-07-01 → 2009-06-25

Expanding window: each fold trains on everything before it and validates on the next 52 weeks. The final 52 weeks of each city are held back entirely.

The constraint is enforced, not just intended

`SARIMAXRegressor.predict()` **raises an exception** if asked to predict a block that is not strictly after its training data:

```

if pd.to_datetime(X["week_start_date"]).iloc[0] <= self.train_end_:
    raise ValueError("SARIMAX needs a future block "
                     "after its training data.")

```

This makes temporal leakage a crash rather than a silently optimistic score. No shuffling, no `cross_val_predict`, anywhere in the project.

Part 2 — Models

The pipeline has four branches

Different model families need different preprocessing, so `make_city_pipeline` builds one of four shapes:

Branch	Steps	Models
<code>tree</code>	impute → history → cyclical → select → model	CatBoost, RandomForest
<code>dense</code>	tree steps + impute + scale	Ridge, Lasso
<code>time_series</code>	impute → history → date + scaled weather	Prophet, SARIMAX
<code>seasonal</code>	calendar columns only	Seasonal median, Shape×level

Why `dense` exists

The `tree` branch leaves **NaNs** in the lag/rolling warm-up — the first 26 weeks have no 26-week history.

- Tree models (sklearn ≥ 1.4) **handle NaN natively**
- RidgeCV raises `ValueError: Input X contains NaN`

So linear models get two extra steps: `SimpleImputer(median)` then `StandardScaler`. Scaling matters for them too, since Ridge and Lasso penalise coefficients and are not scale-invariant.

Why `seasonal` exists

The `tree` branch **drops** `city`, `week_start_date` and `weekofyear` before the model.

But the two calendar baselines need exactly those columns. Rather than weaken the main branch, they get their own: a `ColumnTransformer` passing through `year` and `weekofyear`, nothing else.

These models **never see the weather at all** — which is the point: they are the baseline any weather-driven model should have to beat.

The eight models

Model	Family	Core assumption
CatBoost	gradient boosting	non-linear interactions of lagged weather
RandomForest	bagged trees	same, without boosting
Ridge	linear + L2	smooth linear response, all features shrunk

Model	Family	Core assumption
Lasso	linear + L1	linear, but most features are irrelevant
Prophet	additive time series	trend + annual seasonality + regressors
SARIMAX	ARIMA + exog	autocorrelated errors + Fourier seasonality
Seasonal median	calendar only	this week behaves like this week in past years
Shape $\times$ level	calendar only	seasonal <i>shape</i> $\times$ typical annual <i>total</i>

Results — holdout MAE

Model	San Juan	Iquitos
CatBoost	15.02	6.27
RandomForest	15.16	8.39
Shape $\times$ level	20.72	3.39
Lasso	21.16	4.50
Prophet	21.78	7.16
Seasonal median	23.49	4.41
Ridge	24.46	5.48
SARIMAX	24.56	4.61

Different winners per city — so the final submission uses **CatBoost for San Juan**, **SARIMAX for Iquitos**.

The most interesting result

The best Iquitos model never looks at the weather.

Shape $\times$ level scores **3.39**, beating SARIMAX (4.61) and CatBoost (6.27).

It works in two parts:

1. **Shape** — the seasonal profile, normalised to sum to 1
2. **Level** — the median annual total of complete training years

$$\hat{y}_{\text{week}} = \text{shape}(\text{week}) \times \text{level}$$

Why that result makes sense

The decomposition matches what the EDA found:

- Climate reliably predicts **when** cases rise within a year
- It explains very little about **how many**

Epidemic magnitude is driven by which dengue **serotype** is circulating and by population immunity — neither is in this dataset.

So a model that predicts the *shape* from the calendar and refuses to guess the *level* from weather is not naive. It is **honest about what the data supports**.

Part 3 — Notebook Preparation

Missing values differ by city

	San Juan	Iquitos
ndvi_ne missing	191 / 936 (20.4%)	3 / 520 (0.6%)
ndvi_nw	49	3

San Juan is coastal: the north-east NDVI pixel often falls over the **Atlantic**, where the vegetation index is undefined, and cloud over water blocks the satellite.

One imputation strategy for both cities would be wrong.

DengueImputer — two strategies

Columns	Method	Why
NDVI (San Juan only)	week-of-year training median , then overall median	20% missing in long runs — interpolation would invent a trend
Everything else	linear interpolation, <code>limit_area="inside"</code>	weather is smooth in time; a gap is a sensor outage

`limit_area="inside"` matters: it refuses to extrapolate past the first and last real observation, so we never fabricate data at the edges.

Feature engineering — 20 columns become 124

From 8 weather variables:

- **Lags** at 1, 2, 4, 8, 10, 20, 26 weeks $\rightarrow 8 \times 7 = 56$ columns
- **Rolling means** over 4, 8, 10, 12, 16, 26 weeks $\rightarrow 6 \times 6 = 36$
- **Rolling sums** (rainfall accumulates, it does not average) $\rightarrow 2 \times 6 = 12$

104 engineered columns on top of the 20 originals.

Lags matter because the biology is delayed: weather $\rightarrow$ mosquito breeding $\rightarrow$ infection $\rightarrow$ symptoms $\rightarrow$ diagnosis takes weeks.

The leakage trap in rolling features

A rolling mean computed on the **whole dataset** leaks the future into the past.

`WeatherHistoryTransformer` solves this:

```

def fit(self, X, y=None):
    self.history_ = X.tail(max_history).copy()

def transform(self, X):
    history = self.history_.loc[
        self.history_["week_start_date"] < X["week_start_date"].iloc[0]]
    result = pd.concat([history, X])
    ...
    return result.iloc[-len(X):]

```

Why that design is right

`fit` memorises only the **tail of the training fold**. `transform` prepends only rows **strictly earlier** than the incoming block.

So a validation block gets lag features computed from real past data — not NaN, and not future data.

Every window is also `.shift(1)` before rolling, so **the current week is never included in its own average**.

Why a Pipeline and not inline code

Every preprocessing step is **fitted**: medians, scalers, the feature selector, the history tail.

If preprocessing happens before the split, those statistics are computed on data the model will later be tested on — a subtle leak that inflates every score.

Inside a Pipeline, `GridSearchCV` refits **the whole chain** on each fold's training part only. Correctness becomes structural rather than something to remember.

The refactor — notebook to package

All machinery moved out of the notebook into an importable package:

```

DengAI/
  transformers.py  DengueImputer, WeatherHistoryTransformer
  selectors.py     CatBoostFeatureSelector
  models/         Prophet, SARIMAX, seasonal baselines
  pipelines.py     make_city_pipeline --- the four branches
  evaluate.py      CV, grid search, holdout, submission
configs/
  features.yaml    columns, lags, windows, imputation
  models.yaml      params, grids, branch per model

```

Why configuration lives in YAML

One rule keeps it simple:

Every mapping in the YAML is exactly the `__init__` keyword arguments of the object it configures.

```

WeatherHistoryTransformer(**cfg.features.weather_history)

```

No schema layer, no validation framework. A typo becomes a `TypeError` when the pipeline is built — which is the failure you want, loud and immediate.

Changing the lag grid or adding a model is a config edit, not a code change.

Verification — we proved the move was safe

Before deleting anything from the notebook, the extracted classes were run **side by side** with the originals on real data:

```
sj DengueImputer          IDENTICAL (936, 24)
sj WeatherHistoryTransformer IDENTICAL (52, 128)
iq DengueImputer          IDENTICAL (520, 24)
iq WeatherHistoryTransformer IDENTICAL (52, 128)
```

And the package reproduces the original scores: **SARIMAX San Juan holdout 24.564 vs 24.564 recorded — exact**, Iquitos CV 6.982 exact, identical fold boundaries.

Three real bugs found along the way

1. **The notebook could not run in parallel.** CatBoost's `select_features` ignores `allow_writing_files=False` and does a bare `mkdir("catboost_info")`. With `n_jobs=2`, two workers race and one dies with `FileExistsError`. Fixed with a per-fit `train_dir`.
2. **KeyError: 'catboost'** in the holdout cells — they refit from a dict only populated for models loaded from disk. They now read predictions already computed.
3. **The submission model choice was declared twice, inconsistently.** Now stated once, in `models.yaml`.

Summary

Metrics — MAE because the target is skewed (skewness 4.5); CV MAE tunes, holdout MAE judges; validation is chronological because adjacent weeks correlate at 0.97.

Models — eight models across four pipeline branches, because tree, linear, time-series and calendar families need different preprocessing. Winners differ by city.

Preparation — city-specific imputation, 104 leakage-safe engineered features, and a `Pipeline` so that correctness is structural. All of it extracted into a package with YAML configuration, verified byte-identical before the originals were removed.